

TP – Programmation en PgSQL

Table des matières

1) Problématique.....	1
2) Prérequis	1
3) Exemples	Erreur ! Signet non défini.

1) Problématique

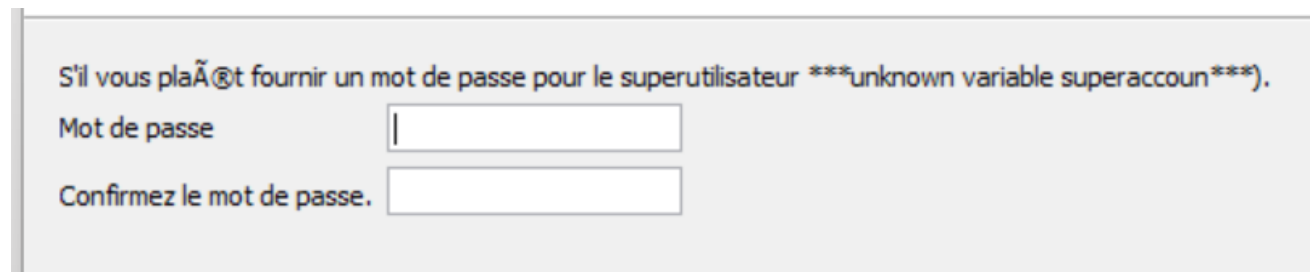
Nous souhaitons apprendre le SQL avec une autre variante de MySQL : PostgreSQL. Nous allons donc répondre à différents exercices en rapport avec cette variante en nous aidant du dossier 3.3 qui nous est fourni.

2) Prérequis

Nous allons devoir installer PostgreSQL pour effectuer les différents exemples du dossier. Pour installer PostgreSQL, que ce soit sur Windows ou Linux, il suffit simplement de se diriger vers le lien suivant afin de choisir la version qui nous intéresse :

<https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>

On nous demande par la suite, lors de l'installation, d'indiquer un mot de passe pour notre utilisateur :



S'il vous plaît fournir un mot de passe pour le superutilisateur `***unknown variable superaccount***`.

Mot de passe

Confirmez le mot de passe.

Dans mon cas je vais utiliser comme mot de passe : **?init01**

Pour le reste on laisse toutes les options par défaut.

A la fin de l'installation, le logiciel StackBuilder s'ouvre, pas besoin de l'utiliser vous pouvez fermer la page.

Ensuite on appuie sur la touche Windows pour chercher « psql » afin de tomber sur le terminal de PostgreSQL. Il faudra cliquer 4 fois sur entrer pour laisser les options par défaut. Ensuite il faudra taper le mot de passe que vous avez choisi, pour moi ça sera donc ?init01

```
SQL Shell (psql)
Server [localhost]:
Database [postgres]:
Port [5432]:
Username [postgres]:
Mot de passe pour l'utilisateur postgres :

psql (18.6)
Attention : l'encodage console (850) diffère de l'encodage Windows (1252).
            Les caractères 8 bits peuvent ne pas fonctionner correctement.
            Voir la section « Notes aux utilisateurs de Windows » de la page
            référence de psql pour les détails.
Saisissez « help » pour l'aide.
```

Pour bien commencer, on va créer notre base de données bac à sable, où l'on y effectuera les différents exemples du dossier.

On utilise donc la commande **CREATE DATABASE nom_bdd** pour créer la base de données et la commande **\c nom_bdd**. Dans mon cas la BDD s'appellera pgsql :

```
postgres=# CREATE DATABASE pgsql;
CREATE DATABASE
postgres=# \c pgsql
Vous êtes maintenant connecté à la base de données « pgsql » en tant qu'utilisateur « postgres ».
pgsql=# |
```

Le terminal est désormais prêt à l'emploi et nous pouvons commencer à effectuer les exercices.

3) Exercices

Exercice 1 : Fonction calculer_longueur_max

On s'aide du dossier fourni pour répondre à cette question, on ajoute les deux paramètres de chaînes : mot1 et mot 2, tous deux VARCHAR. On veut un résultat sous forme d'entier. Ensuite on déclare deux variables longueur, puis on attribue la longueur de chaque mot à un nombre (résultant de la

fonction length(...). Puis on vérifie avec un IF THEN si la longueur du premier mot est plus longue que la longueur du deuxième mot, si c'est le cas alors on affiche la longueur du premier mot, sinon du deuxième.

```
pgsql=# CREATE OR REPLACE FUNCTION calculer_longueur_max(mot1 varchar, mot2 varchar)
pgsql=# RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     longueur integer;
pgsql$#     longueur2 integer;
pgsql$# BEGIN
pgsql$#     longueur := length(mot1);
pgsql$#     longueur2 := length(mot2);
pgsql$#     IF longueur >= longueur2 THEN
pgsql$#         RETURN longueur;
pgsql$#     ELSE
pgsql$#         RETURN longueur2;
pgsql$#     END IF;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT calculer_longueur_max('bonjour', 'salut');
calculer_longueur_max
-----
              7
(1 ligne)
```

Exercice 2 : Nb. D'occurrences d'un caractère dans un intervalle

1. Boucle FOR

On déclare dans la fonction nb_occurences_for 4 paramètres : caractere de type CHAR, cdc de type VARCHAR, debut de type integer et fin de type integer. La fonction retourne u nombre d'occurrence soit un entier donc integer.

On déclare un compteur et i qui représente l'indice. On indique que pour chaque indice de l'intervalle du début à l'intervalle de fin choisi par l'utilisateur, on souhaite en boucle jusqu'à que l'on ait parcouru le dernier indice d'extraire chaque lettre grâce à la fonction **substr** dans lequel on choisit la chaine à analyser, la variable d'indice et la taille de la part que l'on veut couper (ici 1 pour chaque indice). Après on compare la lettre extraite avec le caractère choisi par l'utilisateur et si c'est le même ça augmente de

un le compteur. Puis la fonction retourne la valeur du compteur après la boucle

```
pgsql=# CREATE OR REPLACE FUNCTION nb_occurrences_for(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     compteur integer := 0;
pgsql$#     i integer;
pgsql$# BEGIN
pgsql$#     FOR i IN debut..fin LOOP
pgsql$#         IF substr(cdc, i, 1) = caractere THEN
pgsql$#             compteur := compteur + 1;
pgsql$#         END IF;
pgsql$#     END LOOP;
pgsql$#     RETURN compteur;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_for('n', 'banane', 1, 5);
 nb_occurrences_for
-----
                2
(1 ligne)
```

2. Boucle LOOP

Ici on va utiliser LOOP, on commence par déclarer le compteur et la variable d'indice, puis on donne la valeur de début d'intervalle à l'indice. On utilise le LOOP et on donne la condition de sortie de la boucle qui lorsque i dépasse l'autre bout de l'intervalle (la fin). Ensuite on réutilise le système d'extraction d'avant mais cette fois-ci on doit manuellement indiquer à i de s'incrémenter. Puis comme avant on retourne le résultat du compteur après la boucle.

```
CREATE OR REPLACE FUNCTION nb_occurrences_loop(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
DECLARE
    compteur integer := 0;
    i integer;
BEGIN
    i := debut;

    LOOP
        EXIT WHEN i > fin;

        IF substr(cdc, i, 1) = caractere THEN
            compteur := compteur + 1;
        END IF;

        i := i + 1;
    END LOOP;

    RETURN compteur;
END;
$$ LANGUAGE plpgsql;
FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_loop('n', 'banane', 1, 5);
nb_occurrences_loop
-----
                2
(1 ligne)
```

3. Boucle WHILE

Ici c'est WHILE, on fait comme avant, on attribue la valeur d'intervalle du début à i mais cette fois-ci on utilise WHILE i inférieur ou égal à l'intervalle de fin au lieu de le faire en condition de sortie à l'intérieur de la boucle. Toujours la même fonction d'extraction et comme pour le loop on doit ajouter l'incrémentement de i.

```
pgsql=# CREATE OR REPLACE FUNCTION nb_occurrences_while(caractere char, cdc varchar, debut integer, fin integer) RETURNS integer AS $$
pgsql$# DECLARE
pgsql$#     compteur integer := 0;
pgsql$#     i integer;
pgsql$# BEGIN
pgsql$#     i := debut;
pgsql$#     WHILE i <= fin LOOP
pgsql$#         IF substr(cdc, i, 1) = caractere THEN
pgsql$#             compteur := compteur + 1;
pgsql$#         END IF;
pgsql$#         i := i + 1;
pgsql$#     END LOOP;
pgsql$#     RETURN compteur;
pgsql$# END;
pgsql$# $$ LANGUAGE plpgsql;
CREATE FUNCTION
```

Résultat :

```
pgsql=# SELECT nb_occurrences_while('n', 'banane', 1, 5);
nb_occurrences_while
-----
                2
(1 ligne)
```